Acoplamiento del evaluador y costo por evaluación

TT 2027-A126 · Evidencia 3 del periodo 29 de septiembre – 12 de octubre de 2026

Actividad del cronograma: implementación del motor de optimización, acoplamiento del evaluador con el ciclo del algoritmo y medición del costo por evaluación, para dimensionar el presupuesto de corridas (Echeverría).

Resultado: una evaluación completa cuesta **188 ms en serie y 34 ms en paralelo** (12 procesos). Una corrida de NSGA-II de 25,000 evaluaciones cabe en unos **15 minutos**. El presupuesto no es la restricción del proyecto.

Código: `experimentos/potencial.py` (`evaluar`), `nucleo/optimizar.py` (caché y evaluación en paralelo) y `experimentos/costo_evaluacion.py` (esta medición). Los datos quedan en `datos/costo_evaluacion.json`.

1. Cómo se acopla el evaluador al ciclo

El algoritmo ve al evaluador como una sola función: le da una distribución y recibe todo lo que necesita para compararla.

```
evaluar(recinto, distribución, aforo)
   │
   ├── 1. capa geométrica G1–G6 ........ violación total (0 = válida)   ~2 ms
   ├── 2. zona ocupable ................. dónde hay gente
   ├── 3. f₁ accesibilidad .............. una Eikonal por módulo
   │                                      + transporte capacitado        ~52 ms
   └── 4. f₂ evacuación ................. simulación del fluido hasta
                                          que sale el 95 % (t₉₅)         ~134 ms
   devuelve: f₁, t₉₅, t_des, exposición, violación, tiempo de cada pieza
```

Tres decisiones del acoplamiento:

- **Se calculan las dos funciones aunque se optimice una.** El costo extra es el de f_1, la barata, y a cambio se ve gratis qué le pasa al objetivo que no se está optimizando.
- **Caché por máscara de obstáculos** (`optimizar.Memo`). En las últimas generaciones la población converge y las mutaciones caen sobre distribuciones ya evaluadas. La llave es la rejilla rasterizada: dos distribuciones que ocupan las mismas celdas son el mismo caso.
- **Una generación se evalúa en paralelo** (`ProcessPoolExecutor`). Los individuos de una generación no dependen entre sí, así que se reparten entre procesos; dentro de un lote, cada llave repetida se evalúa una sola vez.

2. Costo medido

Medido el 13 de octubre de 2026 con `py -3.12 experimentos/costo_evaluacion.py --n 96`, sobre 96 distribuciones válidas del recinto de prueba (100 × 60 m, 9 módulos, 2,000 personas), en un AMD Ryzen 5 5600GT con 32 GB.

Pieza	ms	%
capa geométrica	2.4	1.3
f_1 accesibilidad	52.0	27.7
f_2 evacuación	133.7	71.1

Pieza	ms	%
en serie	**188.1**	
en paralelo (12 procesos)	**34.4**	

La simulación es el 71 % del costo. Dentro de ella, la Eikonal se resuelve cada 2.4 s de simulación y no en cada paso de 0.3 s, que es 8 veces más barato. Se verificó que da el mismo orden entre distribuciones (`REFRESCO_CAMPO`).

Antes y ahora. La medición del 23 de septiembre (`experimentos/OPTIMIZADOR.md`, sección 3) dio 400.6 ms en serie y 48.2 ms en paralelo, sobre otra muestra que incluía distribuciones inválidas. Lo que bajó f_1 de 327 a 94 ms en esa fecha fue cambiar el resolvedor de transporte por un descenso por coordenadas con precio exacto por módulo, validado contra el programa lineal exacto (correlación de rangos 1.000000, sin inversiones en 1,225 pares).

3. Presupuesto

Evaluaciones	Tiempo (paralelo)	Qué es
2,448	1.4 min	población 48 × 50 generaciones
10,000	5.7 min	
25,000	14.3 min	población 100 × 250 generaciones

Con esto, cada corrida del siguiente periodo (repeticiones independientes con distintas semillas) cuesta minutos y no horas.

4. Lo que el evaluador ya tiene validado

- **Conservación:** la simulación conserva el número de personas con un error de 5×10^{-15}.
- **Densidad acotada:** con el paso de 0.3 s se cumple la condición CFL en 2D (van Wageningen-Kessels *et al.*, 2018), y la densidad nunca pasa del atasco.
- f_1 **exacta:** el resolvedor por precios da el mismo valor que el programa lineal; se comprueba en `cuadernos/p-mediana.ipynb`.
- **Motor del navegador:** la versión en JavaScript que usa el informe interactivo se compara contra la de Python en `experimentos/validar_js.py`.

5. Mejora pendiente

Con la dominancia restringida, los objetivos de un individuo inválido nunca se consultan: dos inválidos se comparan solo por su violación. Por eso se puede **saltar la simulación cuando la violación es mayor que cero**, lo que ahorra el 98 % del costo de cada hijo inválido. Queda para la integración del siguiente periodo.

Referencias

- van Wageningen-Kessels, F., Daamen, W. y Hoogendoorn, S. P. (2018). Two-Dimensional Approximate Godunov Scheme and What It Means For Continuum Pedestrian Flow Models. *Transportation Science*, 52(3), 547–563. doi:10.1287/trsc.2017.0793